

Progressive Context Control over Typed PRA Records

Jorge Simão

EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 2026

Abstract

Large tool, database, log, graph, retrieval, file, and terminal results should not enter an agent as one permanently visible token stream. We introduce progressive model-controlled context disclosure over typed Parallel Reference Attention (PRA) records. Each result has an exact scoped backing identity and a compact summary registered as an implicit PRA document. Normal PRA routing operates first over summary chunks, gists, and address metadata. If current context is insufficient, a bounded controller may continue, activate the known full record, materialize selected detail, search within that record, advance or query a cursor, or call a new external tool. Every returned detail is another typed view of the same source and re-enters PRA. This separates discovery selection—which record or chunk is relevant—from escalation selection—whether the current materialization is sufficient.

We evaluate a frozen Qwen3-0.6B controller on 30 balanced identities spanning six known context actions, five decoding seeds, and six baselines (900 rows). FULL succeeds on 100.0%, validating every task ceiling. Compact-only succeeds on 16.7%; automatic PRA selection reaches 50.0%; model-only escalation reaches 33.3%; and PRA plus model remains at 50.0%. The combined system uses 99.0 mean materialized tokens versus 266.4 for FULL, but under-expands on 50.0% of rows. A generic retrieve-original CCR baseline reaches 78.0% by spending near-full context and cannot recover information absent from backing state. Thus typed PRA routing economically recovers localized and searchable evidence, while insufficiency detection, cursor use, and tool escalation remain controller bottlenecks. A secondary latent-trigger diagnostic retains the earlier 0.385 to 0.708 improvement, but no longer defines the architecture or primary claim.

1 Introduction

Tool-capable language-model agents increasingly spend their context budget on observations rather than instructions. A database query can return thousands of rows; a build command can produce megabytes of mostly routine output; a graph traversal can expose a neighborhood whose useful branch is not yet known. The agent often needs only a small projection now, but it may need a different projection after its next inference.

Two common responses are incomplete. Full serialization couples active context to source size. One-shot summarization breaks that coupling, but the summary becomes both the model’s visible evidence and its only clue that more evidence exists. An exact original can remain in a cache while being functionally unreachable: the model cannot request a detail whose relevance was itself omitted.

This paper treats context management as a recursive PRA control problem. A result is a typed record, not an undifferentiated message. Its compact view is an implicit PRA document; its full bytes remain exact backing state. PRA first selects useful compact chunks or backing addresses. The model then decides whether to continue or request a bounded escalation. The returned view is typed, linked to the same source identity, and routed again. Large collections may expose cursors instead of repeatedly returning complete snapshots.

The contributions are:

1. a provider-neutral record model for nine result families and a stable mapping from each record to summary, full, selected-detail, search, and cursor-page PRA documents;
2. a scoped content-addressed backing store with exact recovery, hash verification, TTL, size bounds, persistence controls, and audit events;
3. deterministic type-specific compactors and independent lexical, entity, rare-term, and schema address views;
4. a bounded `ContextAction/ContextDecision` API for continue, full, more, search, cursor, and external-tool transitions;
5. stateful cursors supporting pages, ranges, filters, ordering, search, aggregation, sampling, field projection, and closure;
6. a balanced six-class benchmark comparing FULL, compact-only, PRA automatic selection, model escalation, PRA plus model, and CCR retrieval;
7. separate measures of insufficiency recognition, mechanism correctness, task success, evidence recall, context/KV economy, expansions, and transfer;
8. a secondary paired latent-trigger diagnostic and payload-scale transport study, retained without allowing either to define the main system.

The main result is a decomposition, not a blanket win. PRA automatic selection finds evidence that compact-only misses, at a small fraction of full exposure. The small frozen controller does not reliably recognize when full, cursor, or new-tool escalation is required. Paper 7 therefore separates *recovery capability*, *insufficiency detection*, and *mechanism choice*.

2 Context Is a Typed State Space

Let a result record be

$$R = (id, \tau, P, A, V, B), \quad (1)$$

where id is a stable scoped identity, τ is a record type, P contains provenance and policy, A is a set of address views, V is the current prompt-visible projection, and B is lossless backing state. The model need not see A or B directly. They belong to the runtime’s control plane.

This separation prevents three category errors. First, a compact view is not a retrieval index: prose that reads well may be a poor rare-key index. Second, a cache handle is not authorization: identity resolution must still enforce tenant and session scope. Third, storage is not materialization: retaining bytes says nothing about when they enter the model’s active context.

Materialization follows a monotone visibility lattice:

$$V_0 = \text{metadata} \rightarrow V_1 = \text{compact} \rightarrow V_2 = \text{selected fields/range} \rightarrow V_3 = \text{full}. \quad (2)$$

The backing record does not change as a view advances. A selected row range is computed from the hash-verified original after authorization, so an earlier summary cannot silently become the source of truth.

3 Typed Result Records

Table 1 lists the initial result families. The registry is open: applications may replace a compressor without changing identity, storage, authorization, or cursor semantics.

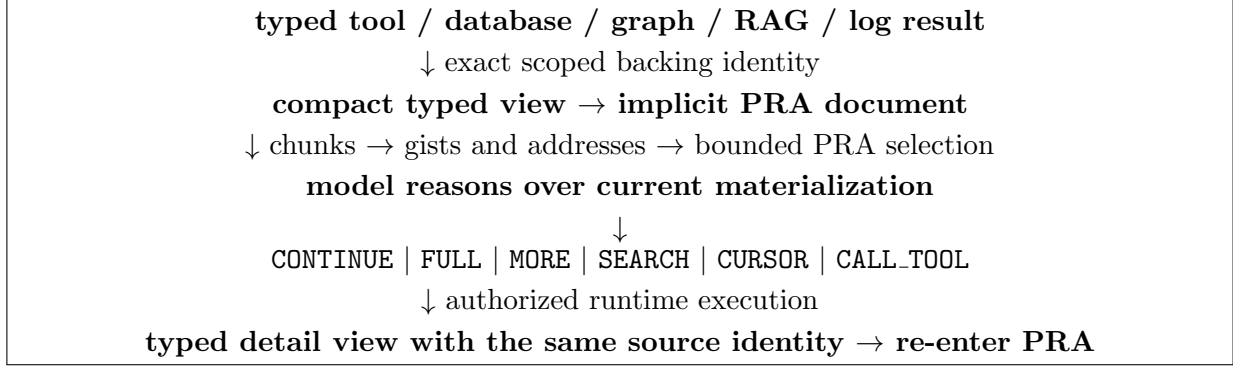


Figure 1: The Paper 7 control loop. PRA performs discovery selection before the model performs escalation selection. Every expansion is an authorized typed view, not model-fabricated content, and recursively re-enters PRA.

Type	Compact view	Useful address/selection units
Tool/API response	bounded structural fields	keys, values, entities, rare identifiers
Log block	errors plus deterministic lines and counts	line ranges, severity, rare tokens
Terminal output	command, status, stderr, stdout head/tail	stdout/stderr ranges, command identity
Database result	schema, representative rows, numeric statistics	fields, row ranges, filters, aggregates
Graph result	node/edge counts and representatives	node IDs, entities, neighborhoods, edge ranges
RAG result/chunks	deterministic passages	document/chunk identity, lexical and entity views
File read	deterministic line projection	path, line ranges, rare keys
Generic text	deterministic line projection	lexical, entity, and rare-term views
Tool/skill capability	inherited selection and full views	stable capability URI

Table 1: Initial type policies. Every lossy row retains an exact original.

3.1 Compact views

The inception implementation uses deterministic compressors because their failure modes are inspectable. Database results retain column names, row count, uniformly spaced representative rows, and min/max/mean statistics for numeric columns. Logs prioritize lines matching error, exception, failure, fatal, panic, denial, and timeout patterns before filling the remaining budget with deterministic samples. Terminal records preserve command, exit status, working directory, stderr, and bounded stdout head/tail. Graph records retain counts and representative nodes and edges. Text-like records sample lines at fixed relative positions.

These policies are baselines, not universal definitions of importance. Error preservation helps logs in the current benchmark, while uniform text sampling sometimes lands on a planted RAG trigger and usually misses it. Optional generated summaries can be registered, but no generated summary is used in the reported study.

3.2 Address views

Address views are built from exact input at ingestion time and stay outside the prompt-visible compact payload. The current implementation emits deterministic lexical tokens, entity-like spans, within-record rare terms, and structural keys. The API reserves dense and native-Q/K views without pretending they are implemented in this checkpoint.

For query q and address view A_j , discovery is

$$\text{score}(q, R) = \sum_j w_j s_j(q, A_j(R)). \quad (3)$$

The inception lexical resolver uses exact normalized token overlap. It is deliberately simple: its role is to test whether omitted bytes remain addressable, not to establish a production semantic retriever.

4 Lossless Backing State

The local store serializes JSON-compatible values canonically and preserves text and byte payloads exactly. A SHA-256 content hash defines the backing identity within a tenant/session scope. Payload and manifest files reside under a hash of that scope. Every read checks the caller scope, TTL, file presence, and content hash before decoding.

The distinction between identity and authorization is operational. Knowing an opaque content-addressed record handle does not permit a different session to read it. Cursor reads repeat the same check. Deletion revokes both payload and manifest; expiration removes them lazily on access or during cleanup. A bounded store evicts oldest entries before accepting further state, and rejects one payload larger than its configured capacity.

The current backend is local filesystem storage. The interface and transport accounting allow a remote or distributed backend later, but this paper does not report a remote storage service, network fault injection, encryption at rest, or multi-process concurrency.

5 Materialization and Transport

Four retrieval modes share one authorization path.

Native typed event. The runtime accepts

$$\text{MATERIALIZE}(id, level, selector, cursor). \quad (4)$$

The identity is already known, so expansion does not rerun semantic search.

Tool retrieval. `retrieve_record` wraps the same event API for providers that expose ordinary function calling. Equal payload and authorization behavior makes the native event an optimization/interface choice rather than a semantic requirement.

Mixed response and cursor. The initial result supplies bounded data, schema, `cursor_id`, `has_more`, and provenance. Later requests fetch only pages, ranges, fields, aggregates, or matching rows.

Proactive expansion. The runtime may materialize a selected projection before the model asks. This can repair a known risk pattern, but it shifts the burden to a policy that must control false positives and transfer cost.

The storage policy is independent of retrieval mode. `upfront` places the original at the model/server side; `on_demand` retains it agent side until expansion; `adaptive` chooses using topology, byte size, and expected reuse. The study evaluates 100,000-byte and 1,000,000-byte thresholds as configuration values. They are not proposed constants.

6 Stateful Cursors

A cursor is a scoped capability over a collection, not a copy of that collection. Its state includes record identity, source, query, schema, collection name, position, page size, filters, ordering, total estimate, creation and expiration times, provenance, authorization fingerprint, and a continuation handle. Supported operations are next/previous page, bounded range, search, filter, numeric aggregate, deterministic sample, selected-field projection, and close.

Cursors matter when the next useful projection depends on the previous result. An analytics agent can request an aggregate, notice an outlying subgroup, and then inspect only rows in that subgroup. A graph agent can inspect one neighborhood before continuing traversal. Repeating a full tool call would discard position and may recompute or mutate source state.

The current cursor manager is in-process and read-only. It does not yet provide transaction isolation, snapshot versioning, distributed leases, or recovery after process loss. Those are serving requirements, not properties inferred from the mechanism tests below.

7 Progressive PRA Context Control

7.1 Implicit documents and source identity

In Paper 7, a compact result is not merely appended as a flat message. For record R , the runtime registers a stable document such as `record://R/summary`. Its serialized typed view is split into bounded chunks; each chunk receives a stable ID, routing gist, lexical/entity address metadata, and the native source identity of R . When a PRA-aware Hugging Face model is attached, registration calls the same `add_reference` path as an ordinary document. The production tokenizer, chunker, hidden-state encoder, gist builder, and native-K/V cache are therefore reused rather than simulated by a result-specific attention path.

Derived documents include `record://R/full`, selected detail, search results, and cursor pages. They share record ID, authorization scope, provenance, and backing fingerprint. Activating `FULL` for a known record is exact identity resolution, not a second global semantic search. This distinction matters operationally: discovery asks which source is relevant; escalation asks how much of that already identified source is needed.

7.2 Bounded decisions

The controller emits a typed decision

$$D = (a, id, s, q, c, t), \quad (5)$$

where a is one action, id a known record, s a typed selector, q an in-record query, c an authorized cursor, and t an external tool. The action space is:

Action	Runtime meaning
CONTINUE	Current PRA-selected context is sufficient.
MATERIALIZE_FULL	Activate the exact bounded backing record.
MATERIALIZE_MORE	Expose declared fields, rows, lines, chunks, or a neighborhood.
SEARCH_RECORD	Search inside one known large backing object.
CURSOR_NEXT/QUERY	Continue or refine an authorized stateful cursor.
CALL_TOOL	Acquire information absent from retained backing state.

The parser rejects unknown record IDs, cursor IDs, selectors, and tools before execution. A local PRA-aware deployment can express D as a native context event. A remote opaque model can use

special retrieval tools. Mixed deployment uses native activation for co-located records and callbacks for remote state. These transports implement the same logical transition.

7.3 Recursive control loop

For query q_t and active typed documents $\mathcal{V}_t$, PRA first computes

$$S_t = \text{PRASelect}(q_t, \mathcal{V}_t; K). \quad (6)$$

The model then emits D_t . If D_t expands state, the runtime authorizes and executes it, produces typed view V_{t+1} , registers that view as an implicit PRA document, and repeats:

$$\mathcal{V}_{t+1} = \mathcal{V}_t \cup \{V_{t+1}\}. \quad (7)$$

The loop has a hard step bound. The model never returns the hidden payload itself; it returns only a decision whose identifiers are resolved by the host.

8 Primary Progressive-Context Experiment

8.1 Questions and protocol

The primary study asks: (RQ1) when is a compact typed view sufficient; (RQ2) can the model recognize insufficiency; (RQ3) what does automatic PRA recover before explicit escalation; (RQ4) does the controller choose full, partial, search, cursor, or tool correctly; and (RQ5) how close does PRA plus model come to FULL at lower exposure cost?

The benchmark contains 30 identities, five in each balanced class:

Class	Required action	Evidence location	Purpose
C0	CONTINUE	compact view	detect needless expansion
C1	FULL	small exact backing	activate known bounded state
C2	MORE	localized row/range	distinguish success from efficient mechanism
C3	CURSOR	later/queryable page	test sufficiency before cursor execution
C4	SEARCH	large known record	test in-record PRA/address retrieval
C5	TOOL	absent from backing	distinguish retrieval from acquisition

Each identity is decoded with seeds $\{11, 23, 37, 53, 71\}$ under six baselines: FULL, COMPACT_ONLY, PRA_AUTO, MODEL_ESCALATION, PRA_PLUS_MODEL, and CCR_TOOL. PRA_AUTO runs bounded address/gist selection but emits no model context decision. MODEL_ESCALATION sees the compact typed view. The proposed PRA_PLUS_MODEL runs automatic selection first and asks the controller only after the returned typed view has re-entered PRA. CCR_TOOL exposes one generic retrieve-original operation. The frozen model is Qwen3-0.6B, using the immutable Ollama blob whose digest begins `sha256-7f4030143c1c` (the full digest is recorded in the manifest). To isolate context-action classification from copying long content-addressed identifiers, the benchmark decoder emits only one action label. The host binds the sole active authorized record or cursor and its preregistered selector, query, or tool. This is narrower than the public API, whose parser validates a complete structured decision against explicit allowlists before execution.

Every identity has an independent FULL answer condition. FULL succeeds on 100.0%, so no class is excluded as a base-model capability failure. We report task success separately from mechanism

Baseline	Task	Evidence	Mechanism	Mat. tokens	Model passes
FULL	1.000	1.000	—	266.4	1.0
COMPACT_ONLY	.167	.167	—	18.4	1.0
PRA_AUTO	.500	.500	.500	31.2	1.0
MODEL_ESCALATION	.333	.333	.167	86.2	2.0
PRA_PLUS_MODEL	.500	.500	.333	99.0	2.0
CCR_TOOL	.780	.780	.780 [†]	258.3	2.0

Table 2: Primary results over 150 rows per baseline. [†]CCR mechanism correctness is binary retrieve/continue correctness, not correct choice among full, partial, search, cursor, and tool.

correctness. Asking for FULL on C2 may answer correctly while still being an inefficient policy. The primary brittleness and waste measures are

$$\text{UnderExpansionRate} = P(\text{required evidence remains hidden}), \quad (8)$$

$$\text{OverExpansionRate} = P(\text{expansion on C0}). \quad (9)$$

Intervals for proportions are Wilson intervals. Paired differences use 10,000 case-cluster bootstrap draws after averaging the five decoding seeds.

8.2 Quality and economy

Table 2 and Figure 2 show three regimes. Compact-only solves exactly C0. PRA_AUTO additionally solves C2 and C4, reaching 50.0% at 31.2 materialized tokens. MODEL_ESCALATION solves C0 and obtains the C2 answer by choosing FULL rather than the required partial mechanism; it misses bounded-full, cursor, search, and external-tool cases. PRA_PLUS_MODEL preserves automatic C2/C4 recovery but does not improve its 50.0% task rate. Its paired gain over compact-only is 0.333 [0.167, 0.500]. The gain is attributable to PRA automatic selection, not autonomous escalation.

CCR_TOOL reaches 78.0% because a generic retrieve-original operation succeeds on C1–C3 and most C4 rows. It materializes 258.3 tokens, near FULL’s 266.4, over-expands every C0 row, and fails C5 because the required information does not exist in retained backing state. This is a useful strong reversible baseline, not the same policy as mechanism-specific escalation.

8.3 Insufficiency and mechanism choice

The proposed system under-expands on 50.0% of all rows and over-expands on 0.0%. Zero over-expansion is not a complete success: it accompanies missed C1, C3, and C5 escalations. Conversely, the model’s C2 use of FULL yields the right answer but wrong mechanism and roughly nine times PRA_AUTO’s C2 materialization (402.0 versus 45.8 tokens). This is why final success and context-policy correctness must be reported separately.

The cursor result is now interpretable. FULL and CCR retrieval succeed on all 25 cursor rows, whereas compact, PRA automatic, model escalation, and the combined policy fail. The cursor manager itself passes deterministic page, range, filter, search, aggregate, scope, expiry, and closure tests. The missing link is the controller’s sufficiency decision, not cursor execution or answer capability.

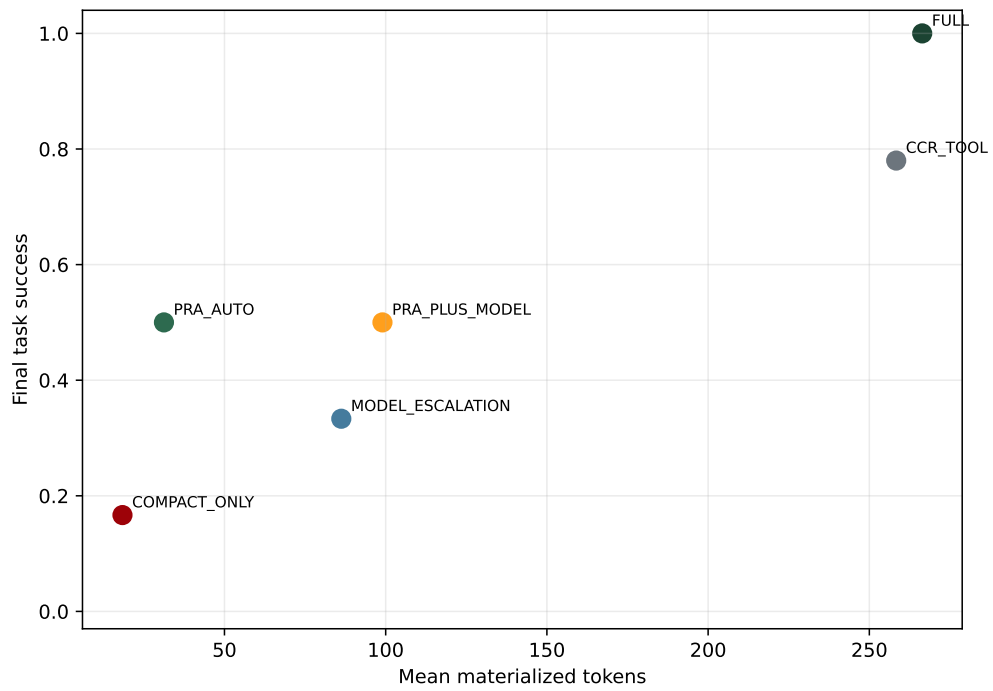


Figure 2: Final task success against mean materialized tokens. FULL is a quality ceiling. PRA automatic selection improves compact-only at low exposure; generic CCR approaches FULL cost and still cannot acquire absent information.

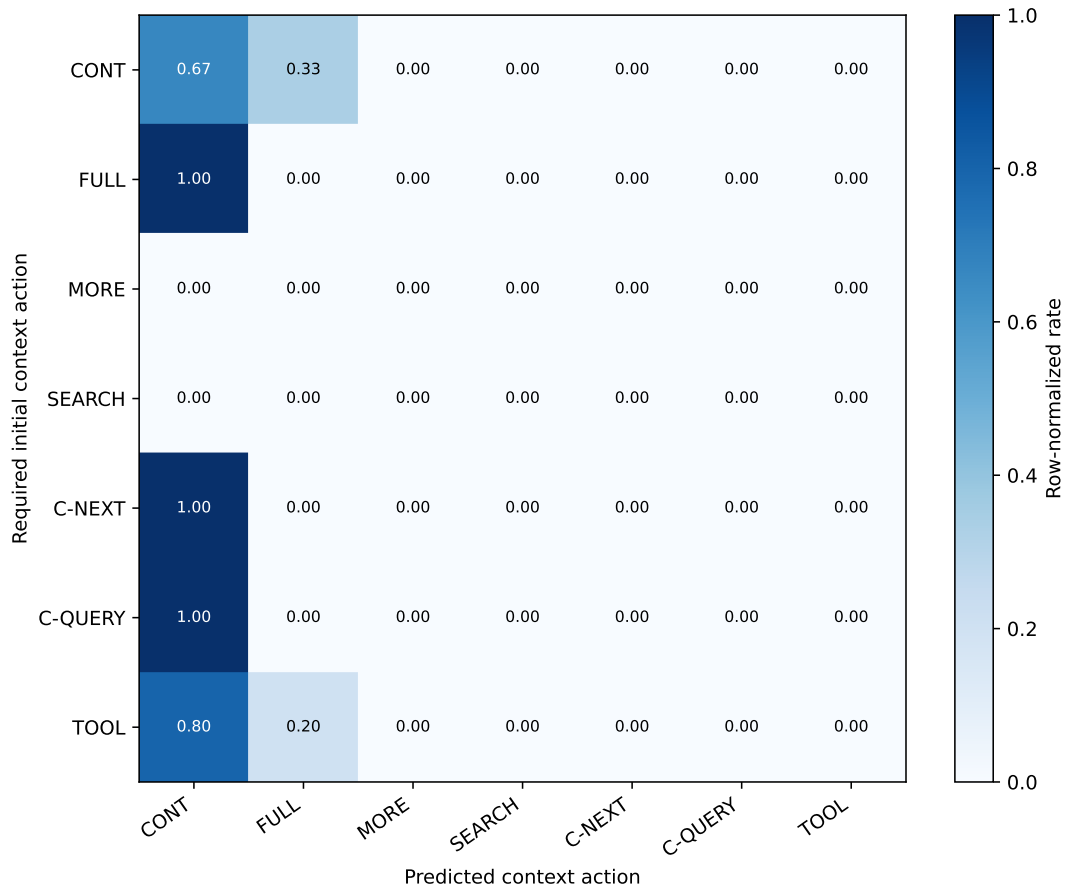


Figure 3: Row-normalized PRA-plus-model context-action confusion. Targets are evaluated after PRA automatic selection; C2 and C4 become CONTINUE when PRA has already exposed their evidence.

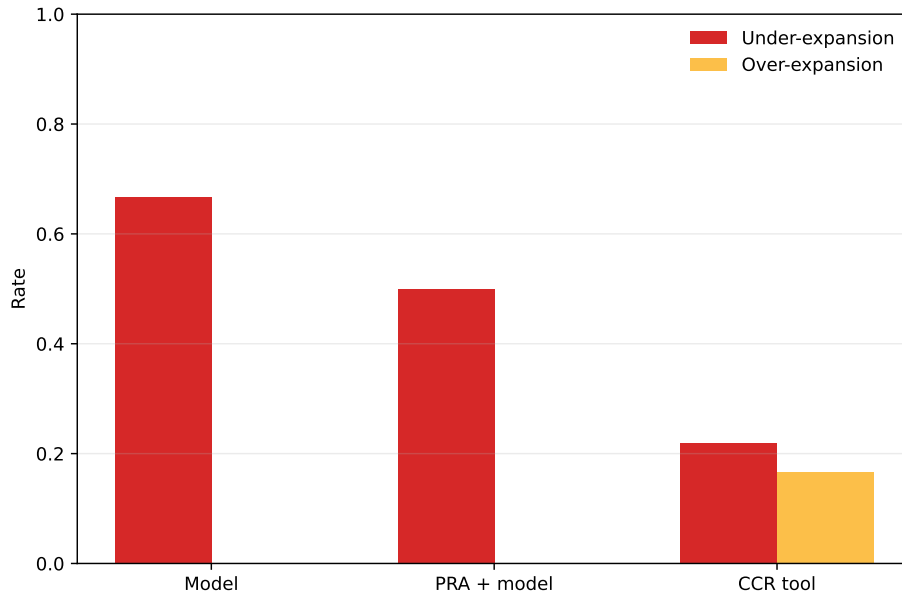


Figure 4: Under-expansion and over-expansion. The small controller is conservative about requesting hidden state; generic CCR trades lower under-expansion for unnecessary C0 retrieval.

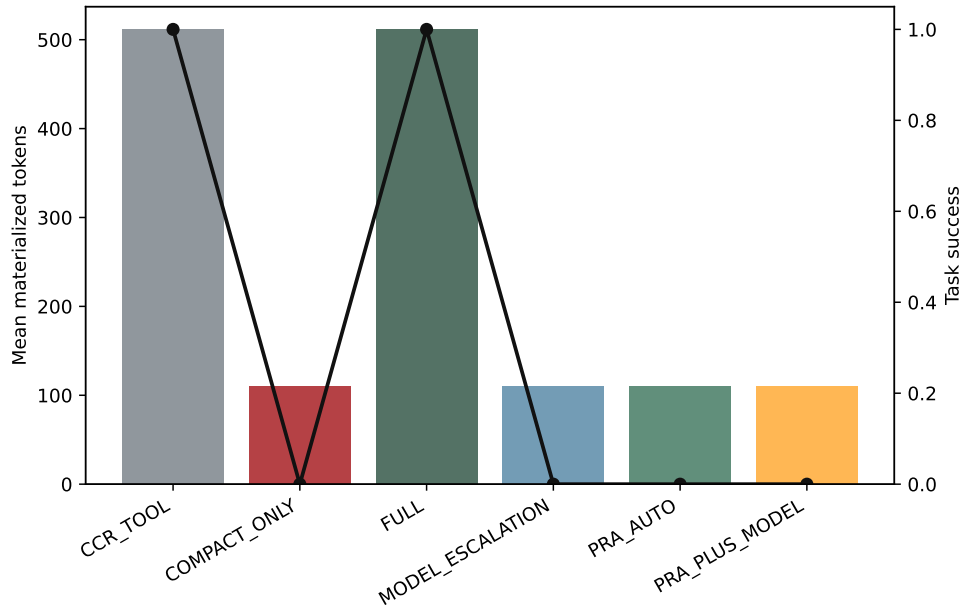


Figure 5: Cursor-class task success and materialized tokens. The repaired FULL and repeated-retrieval ceiling succeeds; the controller does not request the available cursor.

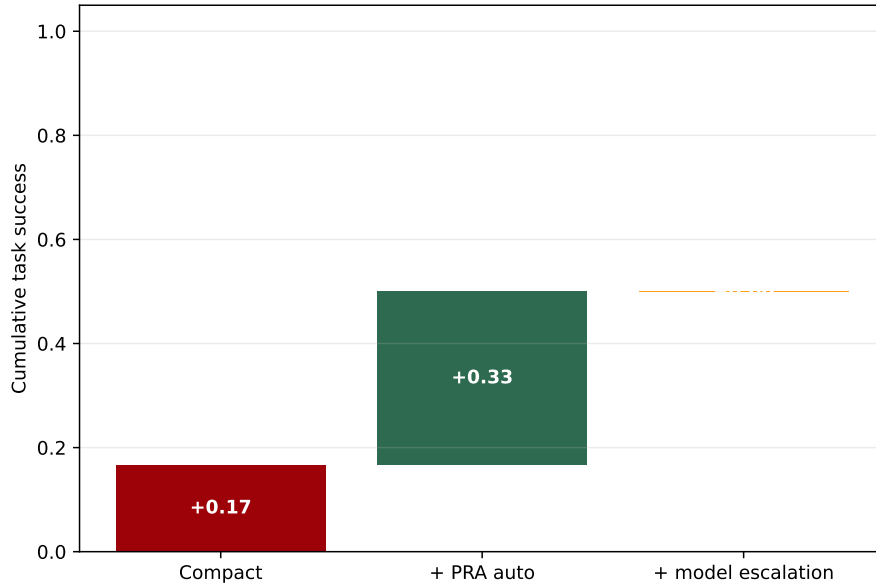


Figure 6: Task-success decomposition. The measured gain over compact-only is provided by PRA automatic selection; this frozen controller adds no net task success after PRA.

9 Why Reversibility Is Not Enough

Storage reversibility answers: “Can the original be recovered if its identity and relevance are known?” Agent behavior asks a harder question: “Will the model realize that an omitted detail should change what happens next?” We separate three cases:

1. **explicit insufficiency**: the query names the hidden concept;
2. **latent insufficiency**: the hidden concept is required but is absent from the query;
3. **action-trigger omission**: the hidden detail would select a different downstream action or tool.

For a compact view V_c , define

$$\text{TriggerRecall} = P(\text{required downstream action remains reachable} \mid V_c). \quad (10)$$

The definition intentionally includes more than answer overlap. A benchmark must observe whether evidence is visible, whether expansion occurs, whether the correct next action remains selectable, and whether an index or proactive policy repairs an omission.

10 Secondary Substrate Mechanism Checks

10.1 Protocol

The study uses seeds {11, 23, 37, 53, 71} and nine result types per seed, giving 45 records. Each payload contains one rare identifier and one required-action token placed away from ordinary high-salience content. Type policies receive a four-unit compact-view budget. No language model, learned compressor, embedding model, or native-Q/K router participates.

We record serialized bytes before and after compression, exact backing equality, trigger presence in the compact view, retrieval by an explicit rare identifier, retrieval by the generic latent query “continue safely,” and retrieval by a proactive probe that names the candidate action. The last condition is an upper-bound mechanism check: it assumes a policy has already generated the correct candidate action. It is not an autonomous latent discovery result.

M4 compares four APIs on the same 100-row database shape. Native-event and tool conditions request full state. Mixed mode opens a five-row cursor page. Proactive mode selects rows 0–4. Reported timing is local Python wall time and is not a model/server latency claim. M5 filters a subgroup, computes its mean, and finds an anomaly through a cursor. M6 executes policy decisions for 10K, 100K, and 1M text payloads at low and high expected reuse across four declared topologies. The topology sweep tests accounting logic; it does not instantiate four physical deployments.

10.2 Compression

Across the nine equally weighted types, mean savings are 0.633. Table 3 reports type means. The graph fixture saves 0.900 and the database fixture 0.797; the small API and tool fixtures save only 0.179 and 0.200. This is expected: fixed metadata can exceed the removable content of a small result. The mechanism should therefore bypass compression below a measured break-even size in production.

10.3 Reversibility is not latent discovery

Figure 8 gives the secondary substrate result. Compact views expose 0.267 of planted triggers. The error-preserving log and terminal policies retain all their triggers; deterministic RAG sampling lands

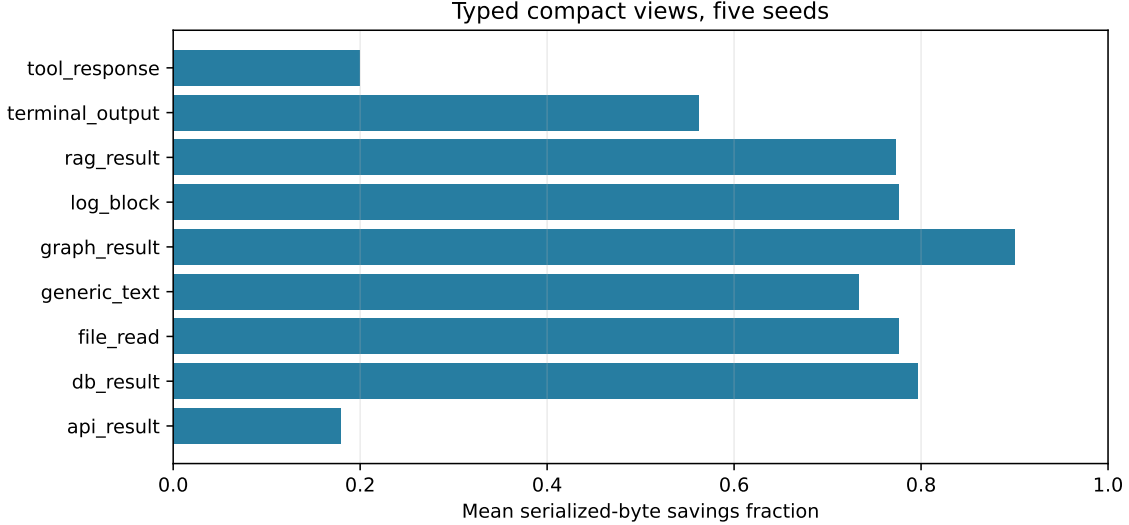


Figure 7: Mean serialized-byte savings by type across five seeds. Small structured tool/API fixtures have limited headroom; graph, database, log, and text-like results are substantially reduced.

Record type	Byte savings	Compact trigger recall
API result	.179	.000
Database result	.797	.000
File read	.776	.000
Generic text	.734	.000
Graph result	.900	.000
Log block	.776	1.000
RAG result	.773	.400
Terminal output	.562	1.000
Tool response	.200	.000

Table 3: Five-seed deterministic mechanism results. Trigger recall measures literal presence in the compact view, not semantic model use.

on two of five; all other compactors miss them. The exact backing store recovers every original, and an explicit rare-token query finds every target through the address index.

The generic latent query finds none. The candidate-action probe finds all, but only because the correct action token is supplied to search. Thus the address layer repairs *omission after a usable discriminator exists*. It does not generate that discriminator. A safe default cannot be justified by reporting only exact recovery or explicit-query recall.

10.4 Retrieval modes and cursors

Native events and retrieval tools return identical full state and transfer 3776 bytes on average. This supports a narrow interface conclusion: a native event is not required for exact semantics when ordinary tool calling is available. The mixed cursor transfers 180 bytes for 0.050 row coverage, while selected proactive materialization transfers 244. Neither partial mode is “better” without a task: both save bytes by returning less data.

The five cursor analytics fixtures obtain 1.000 accuracy for filtered counts and means and 1.000 recall for the planted anomaly. This validates cursor state and selectors. It does not establish that a

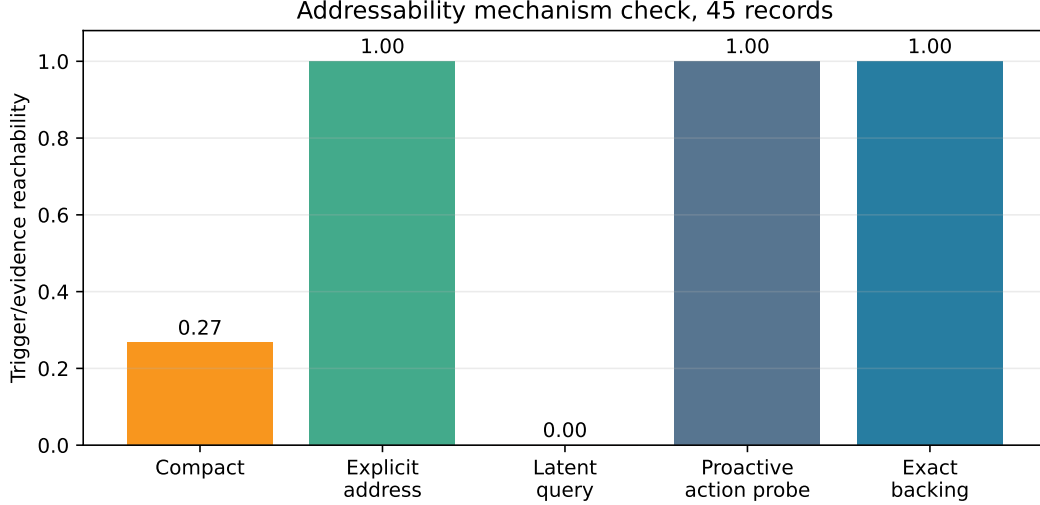


Figure 8: Compact visibility and addressability are different properties. Explicit and candidate-action probes use retrieval-only views; exact backing is hash-verified full recovery.

Mode	Mean transferred bytes	Row coverage	Median local time (ms)
Native full event	3,776	1.00	2.63
Tool full retrieval	3,776	1.00	2.44
Mixed five-row cursor	180	.05	3.21
Proactive five-row selection	244	.05	2.51

Table 4: Mechanism accounting on five seeded 100-row fixtures. These local times exclude model inference and real network transport.

language model will choose the right filter, aggregate, or stopping point.

10.5 Transport policy

The adaptive policy keeps same-process state effectively upfront because no network transfer is needed. For each non-co-located topology, both 10K cases and both 100K cases are upfront under the configured threshold. At 1M, low expected reuse is on-demand and high expected reuse is upfront. This behavior matches the declared rule, but no latency optimum follows from it. A deployment must measure serialization, network bandwidth, backing-store latency, cache reuse, and the probability of expansion before selecting thresholds.

11 Secondary Action-Conditioned Recovery

11.1 Paired protocol

The second study keeps the record substrate fixed and changes the recovery policy. Thirteen deterministic cases cover entity, alias, number/date, rare string, relation, action, state transition, permission, dependency, error code, backend type, schema type, and threshold/anomaly clues. In every case, one hidden field changes the required action. The explicit query names that field; its paired latent query states only the operational goal. The compact view is checked to omit the planted trigger. This is a controlled mechanism benchmark, not a sample of production agent traffic.

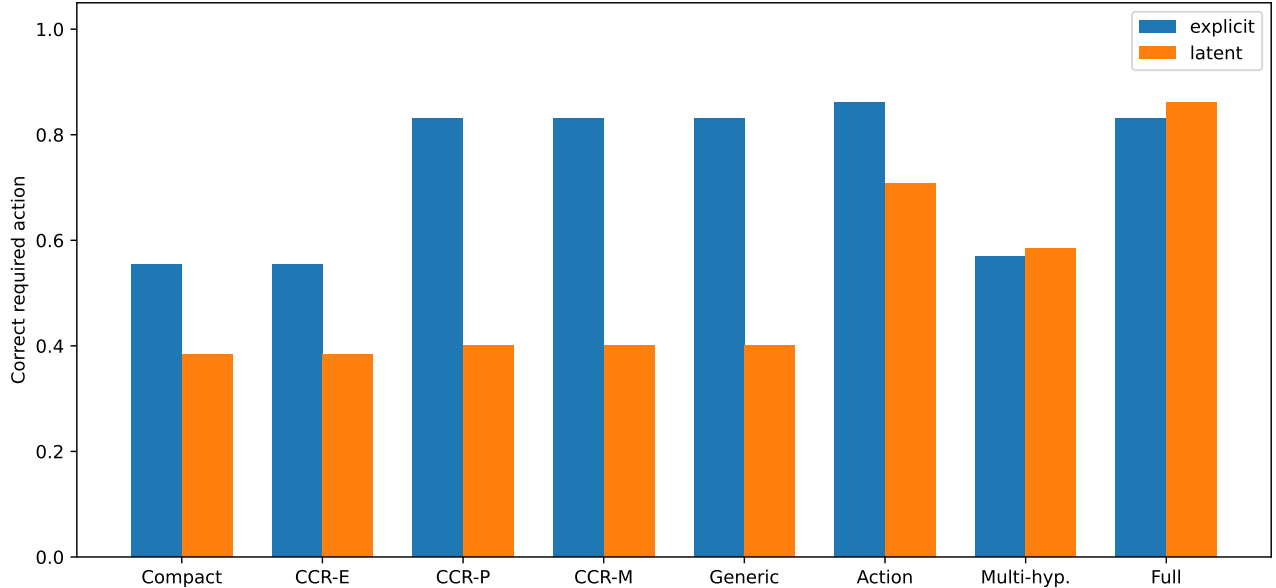


Figure 9: Final required-action accuracy for explicit and latent queries. Full context is a model ceiling rather than a deployable recovery policy. Action-conditioned search improves latent discovery but expands multiple candidate records.

We cross the 13 pairs with eight policies and seeds $\{11, 23, 37, 53, 71\}$, yielding 1040 rows. Candidate-action palettes contain four IDs. The frozen controller is Qwen3-0.6B, Ollama blob `sha256-7f4030...e1fa`, with temperature zero, JSON-constrained output, thinking enabled, and at most four generated hypotheses. A malformed JSON response counts as no decision; it is not repaired with the answer key. Full context is included as a model-capability ceiling.

The policies are compact-only; model-emitted CCR retrieval (CCR-E); proactive lexical CCR expansion (CCR-P); explicit retrieval followed by proactive fallback (CCR-M); generic search over deterministic address views; one probe per candidate action; up to four model-generated hypotheses; and full context. The CCR variants reproduce public architectural behavior—a compact typed view, exact local original, stable marker, retrieval tool, and optional proactive expansion—without claiming implementation identity with any specific system.

We preserve the causal chain

$$\text{goal} \rightarrow \text{hypothesis} \rightarrow \text{address} \rightarrow \text{materialization} \rightarrow \text{action}. \quad (11)$$

TR-address requires a useful hypothesis and target-record address hit; TR-materialize requires exact evidence containing the planted trigger; and TR-action requires the final candidate action. We additionally record false expansions, expansion precision, bytes, estimated tokens, model/tool calls, and local search time. Wilson intervals describe policy-cell proportions. Paired differences use a 10,000-draw bootstrap over 13 trigger cases after averaging the five seeds, so repeated decoding seeds are not treated as new task identities.

11.2 Policy comparison

Compact-only reaches 0.385 TR-action on latent goals. Searching deterministic addresses with the unmodified goal reaches 0.400; reversible storage and a richer index do not create a missing discriminator. Action-conditioned probing reaches 0.708 (46/65; 95% Wilson interval $[0.588, 0.804]$). Its case-clustered

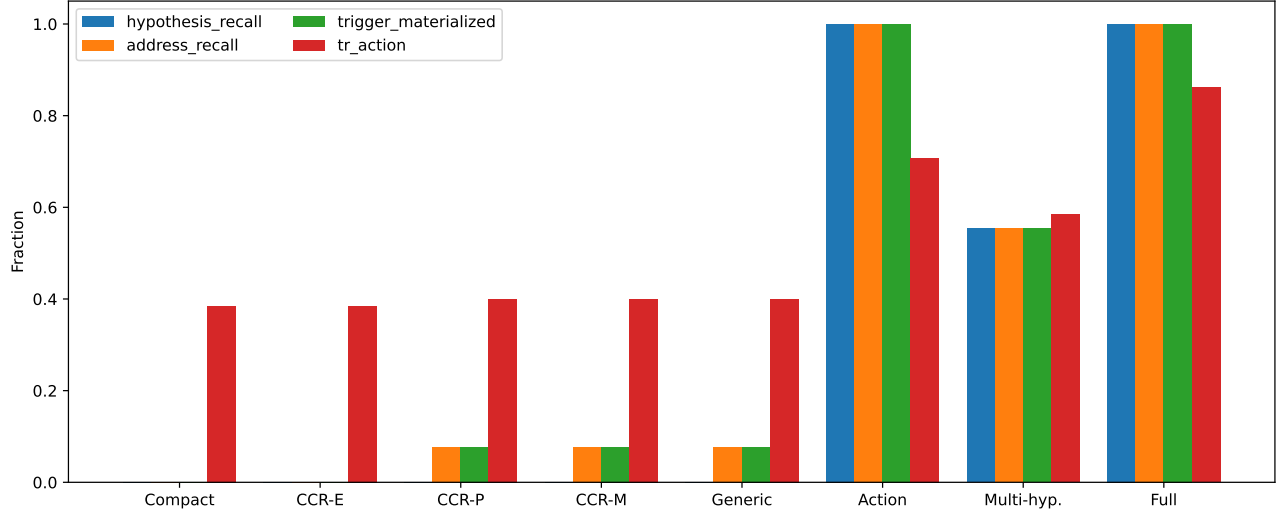


Figure 10: Latent recovery decomposed into hypothesis, address, materialization, and final-action stages. A high later-stage conditional rate cannot compensate for a missing initial hypothesis.

Policy	Hyp.	Materialized	TR-action	Tokens	Exp. precision
Compact only	.000	.000	.385	0.0	.000
CCR-E	.000	.000	.385	0.0	.000
CCR-P	.000	.077	.400	19.2	.077
CCR-M	.000	.077	.400	19.2	.077
Generic proactive	.000	.077	.400	19.2	.077
Action-conditioned	1.000	1.000	.708	643.6	.250
Multi-hypothesis	.554	.554	.585	283.9	.277
Full context	1.000	1.000	.862	251.5	1.000

Table 5: Latent-query means over 13 cases and five seeds. “Hyp.” is hypothesis recall; tokens estimate serialized materialized bytes divided by four. Full context materializes only the target, whereas exhaustive action-conditioned probing may also materialize distractors.

paired difference from compact-only is 0.323 [0.185, 0.477]. The policy materializes 643.6 tokens on average with expansion precision 0.250; it succeeds by spending bounded search and exposure, not by making omission free.

The generated-hypothesis policy reaches 0.585, and full context reaches 0.862. Any gap below the full-context ceiling is attributed to the frozen controller or output protocol. Any policy-to-ceiling gap is then a mixture of discovery, addressing, and expansion errors, which Figure 10 separates.

Table 5 locates the mechanism more precisely. Conditional address and materialization rates are 1.000 whenever action-conditioned or generated hypotheses name the right clue. The generic latent goal generates no such clue and reaches the target in only 5/65 rows through incidental lexical overlap. CCR-E never requests its handle, even in the explicit condition; CCR-P and CCR-M repair explicit omissions but do not solve latent discovery. Malformed or absent model decisions remain failures. This makes the positive claim narrow: bounded action enumeration can recover omitted triggers, but the tested controller does not yet discover them economically.

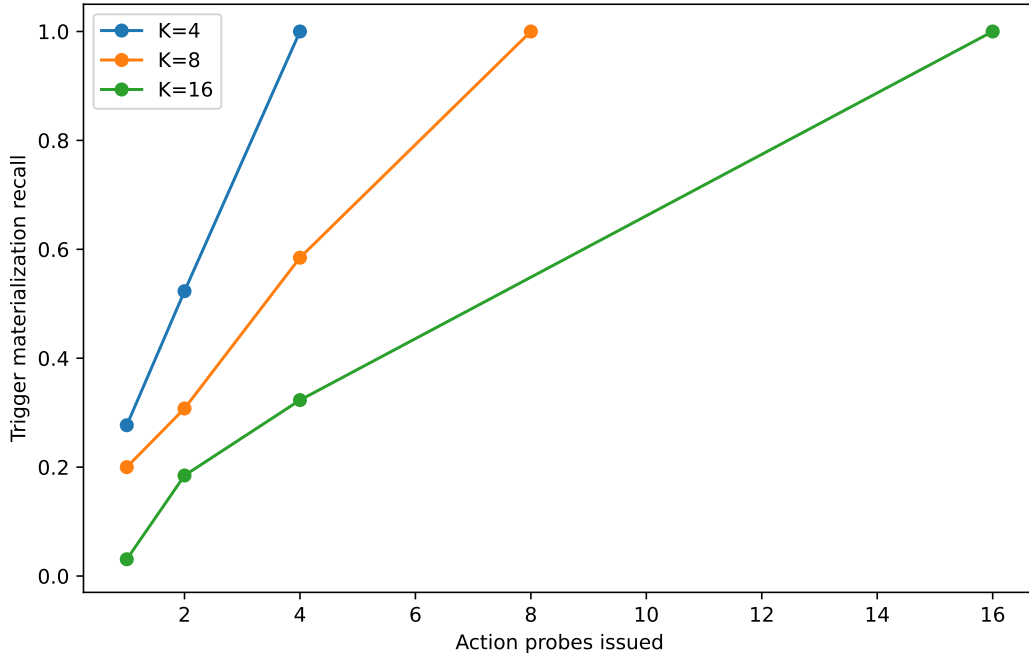


Figure 11: Trigger materialization against probes issued for palettes of $K \in \{4, 8, 16\}$. Exhaustive action probing is an interpretable upper bound; top- r policies trade hypothesis coverage for fewer expansions.

11.3 Probe breadth and address views

The deterministic breadth control varies $K \in \{4, 8, 16\}$ and $r \in \{1, 2, 4, K\}$. Candidate order is seeded but fixed before evaluation. This control asks how often the action that names the hidden evidence enters the probe set; it does not claim that enumeration is an economical controller for unbounded action catalogs. Address ablations compare lexical, entity plus rare-term, schema, lexical plus entity, and all deterministic views. They test whether a useful hypothesis can find its backing record, while keeping hypothesis construction separate.

At $K = 4$, top- r trigger recall rises from .277 at $r = 1$ to .523 at $r = 2$ and 1.000 at $r = 4$. Exhaustive recall remains 1.000 at $K = 8$ and $K = 16$, while precision falls to .125 and .063. With a correct action probe, lexical, entity/rare-term, lexical+entity, and all-view search each recover 13/13 targets; schema-only search recovers 1/13. The richer address layer is therefore adequate after discovery, but it is not a substitute for discovery.

11.4 Quality–economy frontier

The frontier prevents an expand-everything policy from appearing free. On these small fixtures, full target context is both more accurate and cheaper than action-conditioned expansion because the latter materializes three distractors along with the target. Multi-hypothesis uses fewer tokens than action enumeration but still slightly exceeds full-target exposure. The policies are therefore mechanism-positive but not Pareto-superior here. Larger records, selected-field materialization, or better candidate narrowing are required before an economy claim.

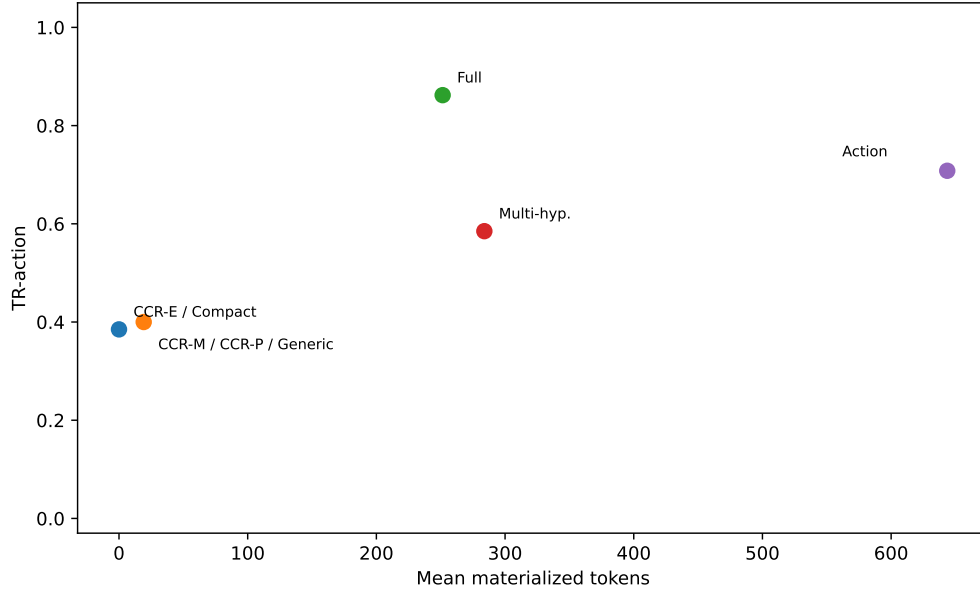


Figure 12: Mean materialized tokens versus latent TR-action. Policies at zero materialization can still exploit compact-view priors; policies farther right pay to expose exact backing evidence.

12 Secondary Transport Accounting

The earlier cursor-answer diagnostic failed even under FULL and is therefore not used for a cursor-quality claim. It motivated the repaired primary C3 tasks above, where FULL and repeated generic retrieval both pass. We retain only its independent transport-policy sweep as a secondary systems check.

Exact recovery is 1.000 in all 36 transport cells. The sweep crosses 10KB, 100KB, 1MB, and 10MB payloads with same-process, separate-local-process accounting, and a remote-like cost model. It evaluates the declared threshold behavior; it is not a measured distributed deployment or a claim that the present thresholds are optimal.

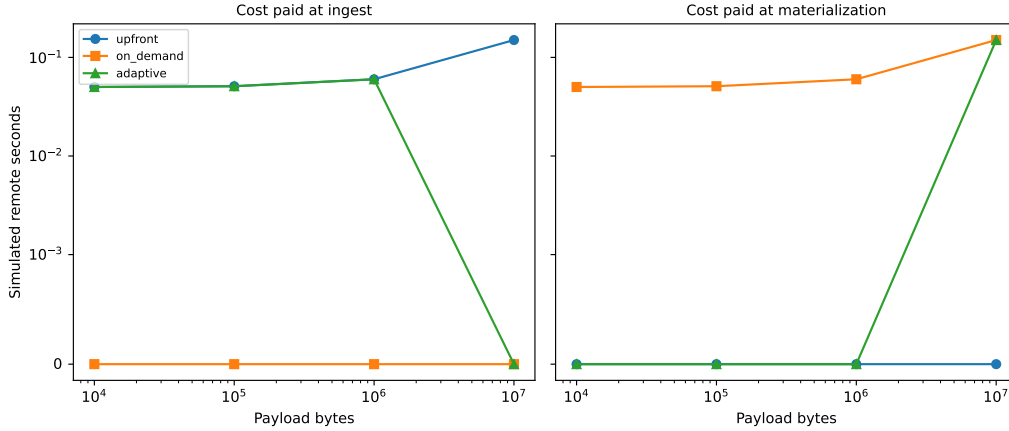


Figure 13: When simulated remote-like transfer cost is paid from 10KB to 10MB. Upfront pays at ingestion, on-demand pays at materialization, and adaptive switches by its declared threshold. RTT and bandwidth components are simulated.

13 Security and Reliability Contract

Typed adaptive context creates capabilities to reveal hidden state. Its security boundary must therefore be explicit.

- Authorization precedes full reads, selected ranges, cursor opens, and every cursor page operation.
- Record IDs and cursor handles are opaque locators, never bearer authorization.
- Tenant and session scope are part of identity resolution; the test suite rejects cross-session reads.
- TTL and explicit deletion revoke state; content hashes detect local payload corruption before decode.
- Audit rows record ingestion, expansion, denial, cursor open, and cursor fetch events with scope and timestamp.
- Redaction is not yet implemented. A production policy must apply it to both compact and full views so expansion cannot bypass disclosure controls.

The store currently writes local plaintext. Encryption, key rotation, distributed authorization, concurrent mutation, cursor revocation propagation, and audit durability remain open production gates.

14 Relationship to Prior Systems

Prompt-compression methods such as LLMingua and LongLLMingua learn or compute shorter prompt representations under token budgets [1, 2]. Paper 7 is orthogonal to the choice of compressor: it specifies where an exact original lives and how different views are addressed and materialized. A learned compressor can be registered without becoming the backing state.

MemGPT frames long-context management as movement between memory tiers and uses explicit control flow to manage what enters context [3]. Paper 7 shares the tiered-state intuition but centers typed tool results, selective fields/ranges, deployment transport, and cursor operations. Retrieval-augmented generation supplies external evidence to a generator [4]; here, a RAG result is itself one typed record that may be compacted, re-addressed, and progressively inspected.

Headroom’s public Compress–Cache–Retrieve documentation describes type-aware tool-output compression, local hash-indexed original caching, a model retrieval tool, markers, and proactive expansion [5]. That architecture is a strong reversible baseline. Paper 7 does not claim that Headroom fails. The primary comparison instead asks whether a typed controller can choose among bounded full, partial, search, cursor, and tool actions rather than exposing only retrieve-or-continue. At nearly full materialization cost, the generic CCR baseline is strong when retained backing state contains the answer, but it cannot acquire absent C5 evidence and does not identify which bounded mechanism was needed. The secondary paired study asks the narrower reachability question: when a compact view omits the fact that would make retrieval relevant, what address or policy keeps the downstream action reachable? Explicit proactive recovery is reliable, generic latent recovery is nearly unchanged from compact-only, and action-conditioned expansion improves the downstream action at substantial exposure cost. These are policy results, not claims about a particular commercial implementation.

Paper 6.5’s typed capability work separates broad tool/skill discovery from bounded selection and exact full-schema activation. Paper 7 preserves those capability records and adds the result side of the interaction: what happens to a typed observation after a tool executes. The companion Paper 3.1 motivates multiple address views by showing that one generated summary can be a brittle retrieval key. The present study tests that design principle with deterministic records rather than repeating its summarization experiments.

15 Limitations and Next Gates

The primary benchmark is a controlled mechanism checkpoint, not a natural agent workload. It has 30 identities and five decoding seeds; seeds measure controller variability, not population diversity. ANSWER_CODE is machine-readable after exposure, which isolates context control but understates interpretation difficulty. The balanced capability descriptors also make the correct mechanism more explicit than many real tool results. Consequently:

1. PRA_AUTO uses transparent lexical/address overlap and one known-record search; learned semantic/native-QK selection may change both quality and cost;
2. the small frozen model is a weak insufficiency detector, so the under-expansion rate characterizes this controller and protocol, not all LMs;
3. CCR_TOOL’s binary mechanism score is not directly comparable to choosing among full, partial, search, cursor, and a new external tool;
4. C3 validates cursor sufficiency decisions and bounded pages, not multi-step transactional analytics or mutable remote snapshots;
5. C5 simulates the returned external tool record after a valid call; it does not evaluate broad tool discovery or side-effect safety;
6. topology measurements are policy accounting, not physical network or distributed-store benchmarks;
7. active native-K/V is estimated from serialized bytes only when a model- specific bytes-per-token value is configured;
8. redaction, remote-store fault tolerance, and distributed cursor leases remain absent.

The next gate is a stronger or calibrated sufficiency controller evaluated on fresh natural tool-result tasks, with class-balanced abstention thresholds and no gold-derived features. It should retain PRA’s

low-cost C2/C4 recovery while adding C1, C3, and C5 escalation without turning every C0 into retrieval. Native PRA-aware events and remote function-call adapters should then be compared at matched semantics while measuring TTFT, active K/V, transfer, callback count, and store latency.

16 Conclusion

Agent context is structured state with a lifecycle, not merely text waiting to be truncated. Paper 7 implements progressive context disclosure over typed PRA records: compact results become routable implicit documents; full bytes retain exact scoped identity; bounded decisions activate full, selected, searched, cursor, or newly acquired state; and each returned view recursively re-enters PRA.

The primary experiment separates what this architecture already solves from what it does not. Automatic PRA selection raises task success from 16.7% to 50.0% while using 31.2 rather than 266.4 materialized tokens. It reliably recovers localized and searchable evidence. The frozen controller adds no net success after PRA and leaves 50.0% of cases under-expanded, especially bounded-full, cursor, and external-tool tasks. Generic CCR retrieval reaches 78.0% by approaching full exposure and cannot acquire absent state. The secondary action-conditioned result shows that missing evidence is recoverable when a useful hypothesis exists, but at distractor cost.

The scientific conclusion is therefore specific. PRA provides an economical discovery substrate for typed backing state; progressive context control adds the correct abstraction for escalation; but a reliable sufficiency controller is still required to connect them. Future gains must improve that controller without erasing PRA’s bounded-context advantage through indiscriminate materialization.

A Public API Sketch

The backing facade keeps deployment policy separate from records. The progressive facade registers compact and derived views through PRA:

```
policy = ContextPolicy(
    storage="adaptive",
    local_store="./tmp/prs",
    retrieval_mode="mixed",
    topology="remote_model",
    record_policies={
        RecordType.DB_RESULT: TypeContextPolicy(unit_limit=8),
    },
    cursor_policy=CursorPolicy(page_size=20, max_page_size=200),
)
backing = AdaptiveContextRuntime(
    RecordScope(tenant_id="tenant-a", session_id="session-42"),
    policy,
)
context = ProgressiveContextRuntime(backing, pra_model=pra_model)
record = context.ingest(
    tool_result,
    record_type=RecordType.DB_RESULT,
    capabilities=RecordCapabilities(
        full_available=True,
        full_bounded=False,
        searchable=True,
        partial_selectors=("rows", "fields"),
    ),
    provenance={"tool": "query_orders", "call_id": "call-17"},
)
```

The model first receives PRA-selected compact chunks. Its JSON is parsed against the active record/cursor/tool allowlists; executing a valid transition registers the returned detail as another typed PRA view:

```

selection = context.automatic_select(
    "failed payments for ACME", top_k=4,
)
decision = parse_context_decision(
    model_json,
    allowed_record_ids=tuple(backing.records),
    allowed_cursor_ids=backing.cursors.cursor_ids,
    allowed_tools=("query_payment_provider",),
)
transition = context.execute(decision, transport="mixed")
# transition.produced_document is now active in the PRA registry.

```

The direct convenience operations share the same path:

```

full = context.materialize_full(record.record_id)
detail = context.materialize_more(
    record.record_id, {"rows": [100, 120]},
)
matches = context.search_record(
    record.record_id, "failed payments for ACME",
)
page = context.cursor_next(cursor_id)

```

B Implementation Map

Module	Responsibility
src/prahf/context_records.py	shared record types and named view vocabulary inherited by capability and result records
src/prahf/context_store.py	scope, content identity, canonical encoding, manifests, TTL, hash verification, capacity, deletion
src/prahf/typed_context.py	compressor registry, compact/address views, adaptive record descriptor, deterministic selectors
src/prahf/adaptive_context_runtime.py	public policy, transport decision, materialization, search, cursor operations, accounting, audit
src/prahf/progressive_context.py	implicit PRA document registry, stable record/view mapping, context actions, decision validation, automatic selection, recursive execution
src/prahf/context_recovery.py	trigger families, CCR-style handles, bounded probes, stage metrics, recovery engine, untrusted cursor-action parser
src/data/progressive_context_cases.py	balanced C0–C5 sufficiency and escalation fixtures
src/data/latent_trigger_cases.py	deterministic paired explicit/latent fixtures and benchmark fingerprint fields
experiments/paper7_records/run_inception_experiments.py	five-seed fixtures, metrics, JSON artifacts, TeX macros, and figures
experiments/paper7_records/run_progressive_context_iteration.py	six-baseline frozen-model matrix, resumable rows, uncertainty, primary plots, and generated TeX
experiments/paper7_records/run_latent_trigger_cursor_iteration.py	frozen-model policy matrix, probe/address controls, cursor tasks, transport sweep, uncertainty, plots, and TeX macros

C Reproducibility

From the repository root:

```
python -m pytest tests/test_context_store.py \
  tests/test_typed_context.py \
  tests/test_adaptive_context_runtime.py \
  tests/test_progressive_context.py \
  tests/test_progressive_context_cases.py \
  tests/test_context_recovery.py -q
python experiments/paper7_records/run_progressive_context_iteration.py
python experiments/paper7_records/run_inception_experiments.py
python experiments/paper7_records/run_latent_trigger_cursor_iteration.py
```

Primary artifacts are under `docs/papers/shared/results/paper7_records/progressive_context`; secondary artifacts are under `docs/papers/shared/results/paper7_records/inception` and `docs/papers/shared/results/paper7_records/latent_triggers`. The manifest records immutable model identity, seeds, decoding, benchmark fingerprint, and simulated-topology labels; local response caches are ignored. Tests cover store identity and integrity, scope and TTL, implicit PRA view identity, real-PRA registration binding, decision validation, recursive view re-entry, accounting, and cursor operations and lifecycle.

References

- [1] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLM_Lingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of EMNLP*, 2023.
- [2] Huiqiang Jiang, Qianhui Wu, Xufang Luo, et al. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *Proceedings of ACL*, 2024.
- [3] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.
- [4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [5] Headroom Labs. Reversible Compression (CCR): Compress–Cache–Retrieve architecture. <https://docs.headroomlabs.ai/docs/ccr>, accessed August 2026.